

Purpose: a beginner-first study document for understanding the project after actually executing it. This is intentionally written as a learning guide rather than a recruiter-facing report.

1. What this project is — in one sentence

This project takes thousands of nested cricket JSON match files, stores them in Snowflake, converts semi-structured JSON into relational tables, builds a small analytical warehouse, and answers business-style cricket questions with SQL.

2. Why we chose this project

It is large enough to demonstrate real data-engineering/analytics thinking rather than only a small CSV dashboard.

The source data is semi-structured JSON, so it forces us to learn Snowflake VARIANT, JSON paths and FLATTEN.

The project naturally demonstrates RAW → STAGING → DW layering, which is useful for Data Analyst, Analytics Engineer and Data Engineer interviews.

The final questions require aggregation, joins, window functions, conditional logic and domain-aware calculations.

It gives a strong fresher interview story: ingestion, transformation, data modeling, validation, analytics and troubleshooting.

3. Key objectives

Ingest the cricket JSON dataset into Snowflake.

Keep the original JSON intact in a RAW layer.

Extract match, team and player information into structured staging tables.

Flatten nested innings → overs → deliveries into one row per delivery.

Build warehouse tables for matches, players, teams and ball-by-ball innings.

Calculate analytical metrics such as runs, wickets, averages and strike rates.

Answer the supplied tournament, T20 and Women's ODI questions.

Understand Snowflake concepts through actual execution rather than only theory.

Document an AWS external-stage attempt and a successful Snowflake-only alternative.

4. The architecture — first understand the picture

The successful hands-on path used in this session was:

```
Cricket JSON files
↓
Snowflake INTERNAL STAGE
↓
RAW.RAW_DATA
↓
VARIANT JSON
↓
FLATTEN + SQL transformations
↓
STAGING tables
↓
DW tables
↓
ANALYTICS queries / views
↓
Power BI later (optional)
```

The original intended cloud architecture was AWS S3 → Snowflake External Stage → Snowpipe → RAW → STAGING → DW. AWS access was blocked by an IAM/STS AssumeRole problem, so we preserved that attempt and completed the warehouse through Snowflake's internal stage instead.

5. The dataset — what is actually inside the JSON?

The JSON is not a flat table. A match contains match-level information under info and nested innings. Each innings contains overs, and each over contains deliveries. A delivery contains batter, bowler, non-striker, runs and sometimes extras, wickets

or reviews.

```
Match JSON
├── info
│   ├── match_type
│   ├── gender
│   ├── dates
│   ├── teams
│   ├── toss
│   ├── outcome
│   ├── event
│   ├── players
│   └── player_of_match
└── innings
    ├── team
    ├── overs
    ├── deliveries
    ├── batter
    ├── bowler
    ├── non_striker
    ├── runs
    ├── extras (optional)
    └── wickets (optional)
```

The uploaded sample JSON demonstrates these nested deliveries, including normal runs, wides, byes/leg-byes and wickets. That is why this is a useful Snowflake semi-structured-data project.

6. What is a Snowflake VARIANT column?

VARIANT is Snowflake's data type for semi-structured data such as JSON. Instead of forcing the JSON into columns immediately, we can store the complete document first and query fields using JSON path syntax.

```
SELECT
  RAW_FILE:info.match_type,
  RAW_FILE:info.gender,
  RAW_FILE:info.teams[0]
FROM CRICKET_ANALYTICS.RAW.RAW_DATA;
```

Beginner mental model: RAW_FILE is the complete JSON document. RAW_FILE:info is the info object. RAW_FILE:info.teams[0] means the first team in the teams array.

7. Why FLATTEN is the heart of the project

Relational tables prefer rows. JSON arrays are nested lists. FLATTEN converts an array into rows so we can work with it using SQL.

```
LATERAL FLATTEN(INPUT => R.RAW_FILE:info.teams)
LATERAL FLATTEN(INPUT => R.RAW_FILE:innings)
LATERAL FLATTEN(INPUT => I.VALUE:overs)
LATERAL FLATTEN(INPUT => O.VALUE:deliveries)
```

The most important transformation is innings → overs → deliveries. The result is one row per delivery. That makes ball-by-ball analysis possible.

8. RAW layer — why keep the JSON?

The RAW layer is the landing zone. We intentionally do not destroy or heavily transform the source document there. This gives us traceability and makes it possible to rebuild downstream tables if a transformation changes.

```
CREATE OR REPLACE TABLE CRICKET_ANALYTICS.RAW.RAW_DATA
(
  RAW_FILE VARIANT,
  FILENAME VARCHAR(100)
);
```

FILENAME is useful because every transformed record can be traced back to the original match JSON file.

9. Internal stage — what changed after AWS failed?

Instead of storing JSON in S3 and asking Snowflake to assume an AWS IAM role, we created a Snowflake internal stage. The files were uploaded directly into that stage through Snowflake.

```
CREATE OR REPLACE FILE FORMAT CRICKET_ANALYTICS.RAW.JSON_FILE_FORMAT
TYPE = JSON;

CREATE OR REPLACE STAGE CRICKET_ANALYTICS.RAW.CRICKET_INTERNAL_STAGE
FILE_FORMAT = CRICKET_ANALYTICS.RAW.JSON_FILE_FORMAT;

LIST @CRICKET_ANALYTICS.RAW.CRICKET_INTERNAL_STAGE;
```

This was not a shortcut that removed Snowflake learning. We still used a stage, file format, COPY INTO, VARIANT, JSON extraction, FLATTEN, staging, warehouse modeling and analytics. Only the external AWS storage dependency was removed.

10. The actual ingestion step

```
COPY INTO CRICKET_ANALYTICS.RAW.RAW_DATA
(
  RAW_FILE,
  FILENAME
)
FROM
(
  SELECT
    $1,
    METADATA$FILENAME
  FROM @CRICKET_ANALYTICS.RAW.CRICKET_INTERNAL_STAGE
)
FILE_FORMAT = (
  FORMAT_NAME = CRICKET_ANALYTICS.RAW.JSON_FILE_FORMAT
)
ON_ERROR = CONTINUE;
```

\$1 means the parsed JSON document from the staged file. METADATA\$FILENAME gives the source file name. COPY INTO is Snowflake's bulk loading command.

Hands-on result: 8,836 JSON files were visible in the internal stage and were loaded into RAW_DATA.

11. STAGING layer — why do we need it?

STAGING is the transformation area between raw source data and the analytical warehouse. Instead of making every dashboard query understand complicated JSON, we flatten and structure the data once.

Main staging tables:

STG_RAW_DATA — copied raw documents for transformation.

STG_TEAMS — one row per team occurrence.

STG_PLAYERS — player/team relationships.

STG_MATCH_INFO — one row per match.

STG_MATCH_INNINGS — one row per delivery.

12. How teams are extracted

```
SELECT DISTINCT
  F.VALUE::STRING AS TEAM_NAME
FROM CRICKET_ANALYTICS.STAGING.STG_RAW_DATA R,
  LATERAL FLATTEN(
    INPUT => R.RAW_FILE:info.teams
  ) F;
```

F.VALUE is the individual array element. ::STRING converts the VARIANT value into a normal string.

13. How players are extracted

The players object is nested by team, so we use two FLATTEN operations: first the team key, then the player array.

```
LATERAL FLATTEN(INPUT => R.RAW_FILE:info.players) T,
LATERAL FLATTEN(INPUT => T.VALUE) P
```

T.KEY gives the team name; P.VALUE gives the player name. This produces a player-team relationship table.

14. Match information transformation

One JSON file represents one match, so match-level fields can be extracted directly from RAW_FILE:info.

```
R.RAW_FILE:info.match_type::STRING
R.RAW_FILE:info.gender::STRING
R.RAW_FILE:info.dates[0]::DATE
R.RAW_FILE:info.teams[0]::STRING
R.RAW_FILE:info.teams[1]::STRING
R.RAW_FILE:info.toss.winner::STRING
R.RAW_FILE:info.toss.decision::STRING
R.RAW_FILE:info.outcome.winner::STRING
R.RAW_FILE:info.event.name::STRING
```

The outcome logic converts either a runs margin or wickets margin into a readable margin field.

15. Ball-by-ball transformation — the most important SQL

The delivery transformation is where the nested JSON becomes a relational fact-style table. Conceptually we perform:

```
match
→ innings
→ overs
→ deliveries
```

For every delivery we capture:

```
inning_id
over
batting_team
ball_in_over
batter
bowler
non_striker
runs_batter
extra_type
runs_extras
runs_total
dismissal_kind
out_player
is_powerplay
filename
```

16. Extras and wickets — why this matters

Cricket statistics cannot be calculated correctly by treating every delivery as a simple run. Wides, no-balls, byes and leg-byes behave differently for batting and bowling calculations.

```
CASE
WHEN D.VALUE:extras.wides IS NOT NULL THEN 'wides'
WHEN D.VALUE:extras.noballs IS NOT NULL THEN 'noballs'
WHEN D.VALUE:extras.byes IS NOT NULL THEN 'byes'
WHEN D.VALUE:extras.legbyes IS NOT NULL THEN 'legbyes'
ELSE NULL
END AS EXTRA_TYPE
```

Wickets are read from the optional wickets array. If there is no wicket, the field remains NULL.

17. Powerplay logic

The project uses powerplay information where available and keeps default configurations for common ODI and T20 windows. A delivery is marked as powerplay when its over falls inside the applicable start/end range.

```
CASE
WHEN OVER BETWEEN POWERPLAY_START AND POWERPLAY_END
THEN 1
ELSE 0
END AS IS_POWERPLAY
```

18. Data Warehouse layer

The DW layer contains the main analytical tables:

19. Why sequences were used

Sequences generate numeric IDs for teams, players and matches. This separates warehouse identifiers from source names.

```
CRICKET_ANALYTICS.RAW.SEQ_TEAMS.NEXTVAL
CRICKET_ANALYTICS.RAW.SEQ_PLAYERS.NEXTVAL
CRICKET_ANALYTICS.RAW.SEQ_MATCHES.NEXTVAL
```

20. Why the player table stores an ARRAY

A player may be associated with more than one team. Instead of duplicating the player master row for every team, the project stores team IDs as an ARRAY and can FLATTEN that array when needed.

21. Analytics questions the project answers

Player statistics: runs, wickets, batting average, bowling average, batting strike rate and bowling strike rate.

Highest run scorer for each event/tournament.

Highest wicket taker for each event/tournament.

Highest T20 powerplay strike rate.

Maximum sixes in T20 overs 17–20.

International players with centuries.

Top 5 Women's ODI run scorers.

Top 5 Women's ODI wicket takers.

22. Example: batting strike rate

The basic formula is:

$$\text{Batting Strike Rate} = (\text{Runs} / \text{Balls Faced}) \times 100$$

In SQL, we count balls faced while excluding wides, then use DIV0 to avoid division-by-zero errors.

23. Example: bowling strike/rate logic

The project calculates bowling metrics from legal balls, runs conceded and credited wickets. The exact cricket-statistic definition must always be defended with the assumptions used in the SQL.

24. Window functions — why they appear

RANK() is used when we want the leader inside every tournament/event, not merely the overall leader.

```
RANK() OVER (  
  PARTITION BY EVENT_NAME  
  ORDER BY TOTAL_RUNS DESC  
) AS RUN_RANK
```

PARTITION BY creates a separate ranking group for every event. ORDER BY determines the ranking inside each group.

25. Streams and Tasks — why they were not the first focus

The first successful run deliberately focused on proving the core pipeline manually. Streams and Tasks do not reduce the amount of transformation logic; they automate when and how that logic runs.

They are a useful V2 enhancement. We intentionally did not make them a prerequisite for understanding or proving the core project.

26. AWS attempt — what happened

Original intended path:

```
AWS S3 → External Stage → Snowpipe → RAW
```

Known errors encountered:

Storage integration creation initially failed because STORAGE_AWS_ROLE_ARN was required.

After configuring an IAM role, Snowflake still returned an STS AssumeRole authorization error.

SYSTEM\$VALIDATE_STORAGE_INTEGRATION was initially called with the wrong number of arguments; the four-argument form was then used.

A LIST statement was briefly malformed because the stage reference was concatenated with unrelated text; the clean LIST @stage statement worked.

The important learning is not that AWS was 'bad'; it is that external cloud integrations introduce IAM, trust-policy, external-ID and role-assumption dependencies. For this learning project, we isolated that blocker and continued with Snowflake's internal stage.

27. What we actually achieved hands-on

Extracted the nested ZIP archives locally so Snowflake received actual JSON files.

Uploaded the full dataset through the Snowflake internal stage.

Verified 8,836 JSON files in the stage.

Loaded the JSON documents into RAW_DATA.

Verified a complete JSON document inside Snowflake's VARIANT column.

Executed more than 1,400 lines of project SQL.

Reported approximately 95% successful execution in the first full run.

Reached the staging, warehouse and analytics layers; remaining failed statements should be reviewed individually rather than blindly rerun.

28. What the beginner should learn during the review week

29. The interview explanation — beginner version

"I worked on a cricket analytics project in Snowflake using thousands of JSON match files. The main challenge was that the data was nested rather than tabular. I stored the JSON in a RAW VARIANT table, used FLATTEN to convert innings, overs and deliveries into relational staging data, and then built warehouse tables for matches, players, teams and ball-by-ball data. I used SQL aggregations and window functions to answer player, tournament, T20 and Women's ODI analytics questions. I initially attempted an AWS S3 external-stage architecture, but an IAM/STS AssumeRole issue blocked the connection, so I implemented the same Snowflake warehouse pipeline using an internal stage. That gave me hands-on experience with both the architecture and troubleshooting."

30. What makes this a strong fresher project?

Semi-structured data instead of a simple CSV.

Large number of source files.

Cloud data warehouse experience.

Layered data architecture.

JSON parsing and FLATTEN.

Business-style analytical questions.

Data-quality/validation checks.

Real troubleshooting and alternative implementation.

Clear path to Power BI and later dbt/automation.

31. What not to claim in an interview

Do not say AWS S3 ingestion was successfully automated; it was attempted but blocked by IAM/STS.

Do not say Streams/Tasks were production-tested if they were not.

Do not claim every one of the 1,400+ statements succeeded; the first run was approximately 95% successful.

Do not claim advanced dbt, Airflow, Spark or Databricks implementation unless separately completed.

Do not memorize SQL without understanding the purpose of each layer.

32. Future upgrades

Add a controlled Streams + Tasks automation layer.

Add dbt staging/mart models.

Connect curated Snowflake tables to Power BI.

Create a cricket analytics dashboard.

Add tests for nulls, duplicate matches and referential consistency.

Keep AWS S3 as a documented optional external ingestion architecture once IAM is correctly configured.

33. Final mental model

```
RAW answers:      "What did we receive?"
STAGING answers:  "How do we turn JSON into usable rows?"
DW answers:       "How do we organize the data for analytics?"
ANALYTICS answers: "What can we learn from the data?"

VARIANT = stores JSON
FLATTEN = opens nested arrays
COPY INTO = loads files
JOIN = connects entities
GROUP BY = summarizes
WINDOW FUNCTION = ranks/calculates within groups
VIEW = reusable analytical query
STREAM = detects changes
TASK = automates SQL
```

If you understand those ten ideas, you understand the project at a practical beginner level. The goal of the review week is to move from “I executed this with help” to “I can explain why every major part exists.”

34. Code appendix strategy

The full executable worksheet is intentionally kept in Snowflake as 04_SNOWFLAKE_ONLY_ALTERNATIVE. This learning report explains the logic rather than duplicating all ~1,400 lines. During review, use the worksheet and this report together: read one section, run the corresponding SQL, inspect the result, and explain it in your own words.

Status: Hands-on Snowflake-only pipeline completed to the point reported on 4 October 2026. Remaining failed statements should be diagnosed individually during the review session before calling the implementation fully polished.

Table	Role	Beginner meaning
DW.TEAMS	Team dimension	Master list of teams
DW.PLAYERS	Player dimension	Player identity + team relationships
DW.MATCH_INFO	Match dimension	One row per match
DW.MATCH_INNINGS	Ball-by-ball fact	One row per delivery

Concept	What it does	Mental model
Stream	Tracks table changes/new rows	Change detector
Task	Runs SQL on a schedule/dependency	Automation engine
Task graph	Controls execution order	Production workflow

Review	Focus
Day 1	Understand database/schema/table/stage/file format and COPY INTO.
Day 2	Understand VARIANT, JSON paths and FLATTEN.
Day 3	Rebuild STG_TEAMS, STG_PLAYERS and STG_MATCH_INFO mentally.
Day 4	Deeply understand innings → overs → deliveries.
Day 5	Understand DW tables, keys, sequences and joins.
Day 6	Understand the analytics queries, RANK(), GROUP BY and cricket metrics.

Review	Focus
Day 7	Explain AWS attempt, internal-stage alternative, Streams/Tasks and the interview story.